Experiment 16: LSTM and RNN Performance Comparison

Question

Problem Statement: A food-delivery platform wants to forecast daily order volume for the next week. Long-term weekly and seasonal patterns matter. Two years of daily order counts with weekly and festival-season patterns (or an airline passenger dataset). The objective is to compare SimpleRNN and LSTM on a long-dependency forecasting task. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Prepare sequences with a 30-day look-back window.
2. Train a SimpleRNN and an LSTM with the same number of units.
3. Compare training loss, validation loss and training time.
4. Compute MAE, RMSE and MAPE on the test set.
5. Plot both forecasts against actual values; tabulate the results with Pandas.

Code

```
"""Experiment 16: LSTM and RNN Performance Comparison (daily food-order forecasting)"""
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

np.random.seed(42); tf.random.set_seed(42)

# ---------------- Data: 2 years of daily orders with weekly + festival patterns ----------------
days = pd.date_range("2023-01-01", periods=730, freq="D")
t = np.arange(730)
orders = (1000 + 0.3 * t                                                    # growth trend
          + 250 * (days.dayofweek >= 5)                                      # weekend peak
          + 400 * np.exp(-((t % 365 - 300) ** 2) / (2 * 8 ** 2))           # festival season (~day 300)
          + 200 * np.exp(-((t % 365 - 100) ** 2) / (2 * 6 ** 2))           # second festival (~day 100)
          + np.random.normal(0, 40, 730))
df = pd.DataFrame({"orders": orders}, index=days)
# For the airline passengers dataset use:  df = pd.read_csv("airline-passengers.csv", index_col=0)
print(df.head())

split = int(0.8 * len(df))
scaler = MinMaxScaler().fit(df.iloc[:split])
scaled = scaler.transform(df).ravel()

# ---------------- Task 1: 30-day look-back window ----------------
WINDOW = 30
X, y = [], []
for i in range(len(scaled) - WINDOW):
    X.append(scaled[i:i + WINDOW]); y.append(scaled[i + WINDOW])
X = np.array(X)[..., None]; y = np.array(y)
cut = split - WINDOW
X_train, y_train, X_test, y_test = X[:cut], y[:cut], X[cut:], y[cut:]
print("Train:", X_train.shape, "Test:", X_test.shape)

# ---------------- Task 2: same number of units ----------------
UNITS, EPOCHS = 64, 60
def build(cell):
    m = keras.Sequential([layers.Input((WINDOW, 1)), cell(UNITS), layers.Dense(1)])
    m.compile(optimizer=keras.optimizers.Adam(0.005), loss="mse")
    return m

results, preds, histories = [], {}, {}
actual = scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()
for name, cell in [("SimpleRNN", layers.SimpleRNN), ("LSTM", layers.LSTM)]:
```

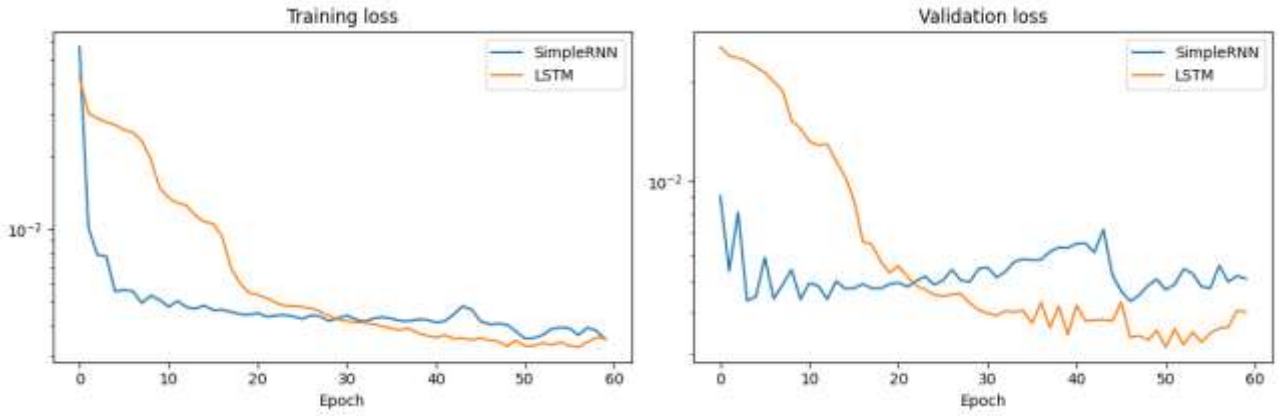
```
    tf.random.set_seed(42)
    model = build(cell)
    t0 = time.time()
    h = model.fit(X_train, y_train, validation_split=0.15, epochs=EPOCHS, batch_size=32, verbose=0)
    train_time = time.time() - t0
    p = scaler.inverse_transform(model.predict(X_test, verbose=0)).ravel()
    mae = mean_absolute_error(actual, p)
    rmse = np.sqrt(mean_squared_error(actual, p))
    mape = np.mean(np.abs((actual - p) / actual)) * 100                  # Task 4
    results.append({"Model": name, "Params": model.count_params(), "Train loss": h.history["loss"][-1],
                    "Val loss": h.history["val_loss"][-1], "Train time (s)": round(train_time, 1),
                    "MAE": mae, "RMSE": rmse, "MAPE (%)": mape})
    preds[name] = p; histories[name] = h.history
    print(f"{name:10} time={train_time:5.1f}s  MAE={mae:7.2f}  RMSE={rmse:7.2f}  MAPE={mape:5.2f}%")

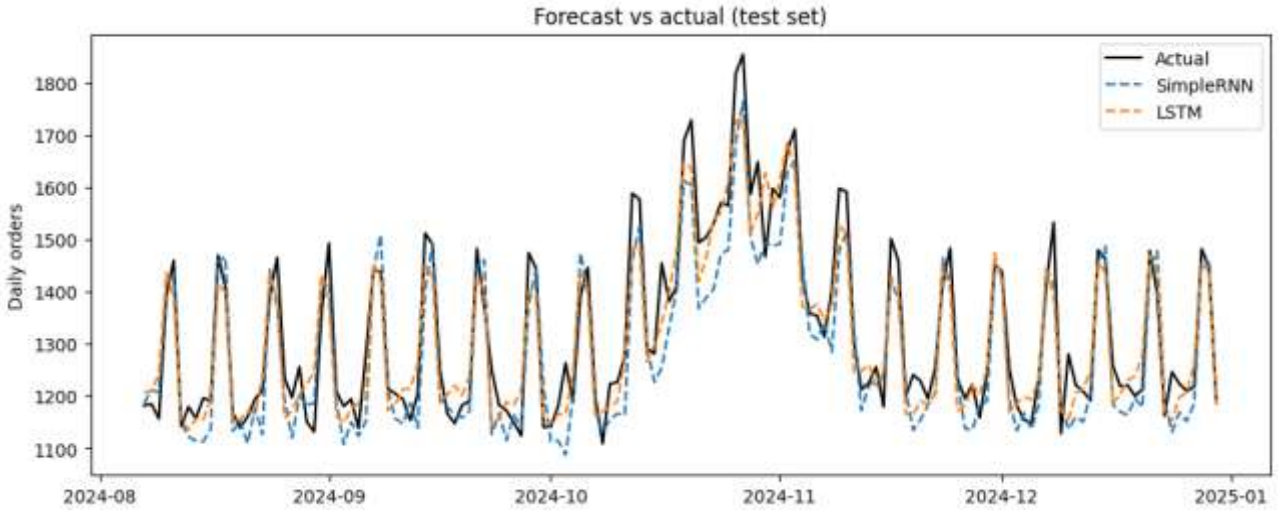

# ---------------- Task 3: loss comparison ----------------
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for name, h in histories.items():
    ax[0].plot(h["loss"], label=name); ax[1].plot(h["val_loss"], label=name)
ax[0].set_title("Training loss"); ax[1].set_title("Validation loss")
for a in ax:
    a.set_xlabel("Epoch"); a.set_yscale("log"); a.legend()
plt.tight_layout(); plt.show()

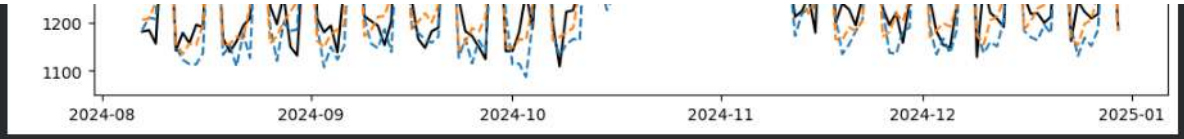
# ---------------- Task 5: forecast plot and results table ----------------
plt.figure(figsize=(12, 4.5))
plt.plot(df.index[-len(actual):], actual, "k", label="Actual")
for name, p in preds.items():
    plt.plot(df.index[-len(actual):], p, "--", label=name)
plt.ylabel("Daily orders"); plt.title("Forecast vs actual (test set)"); plt.legend(); plt.show()

table = pd.DataFrame(results).set_index("Model")
pd.set_option("display.width", 200); pd.set_option("display.max_columns", None)
print("\n", table.round(4))
```

Output







Model	Params	Train loss	Val loss	Train time (s)	MAE	RMSE	MAPE (%)
SimpleRNN	4289	0.0035	0.0051	32.4	54.8419	67.9245	4.0607
LSTM	16961	0.0035	0.0040	26.5	45.3506	55.6050	3.3869